

Asynchronous FIFO Design & UVM Verification Environment

Hardware RTL & Functional Verification Report

September 18, 2026

RTL: VHDL-2008

Verification: SystemVerilog

Methodology: UVM 1.2

Simulator: QuestaSim / ModelSim

Table of Contents

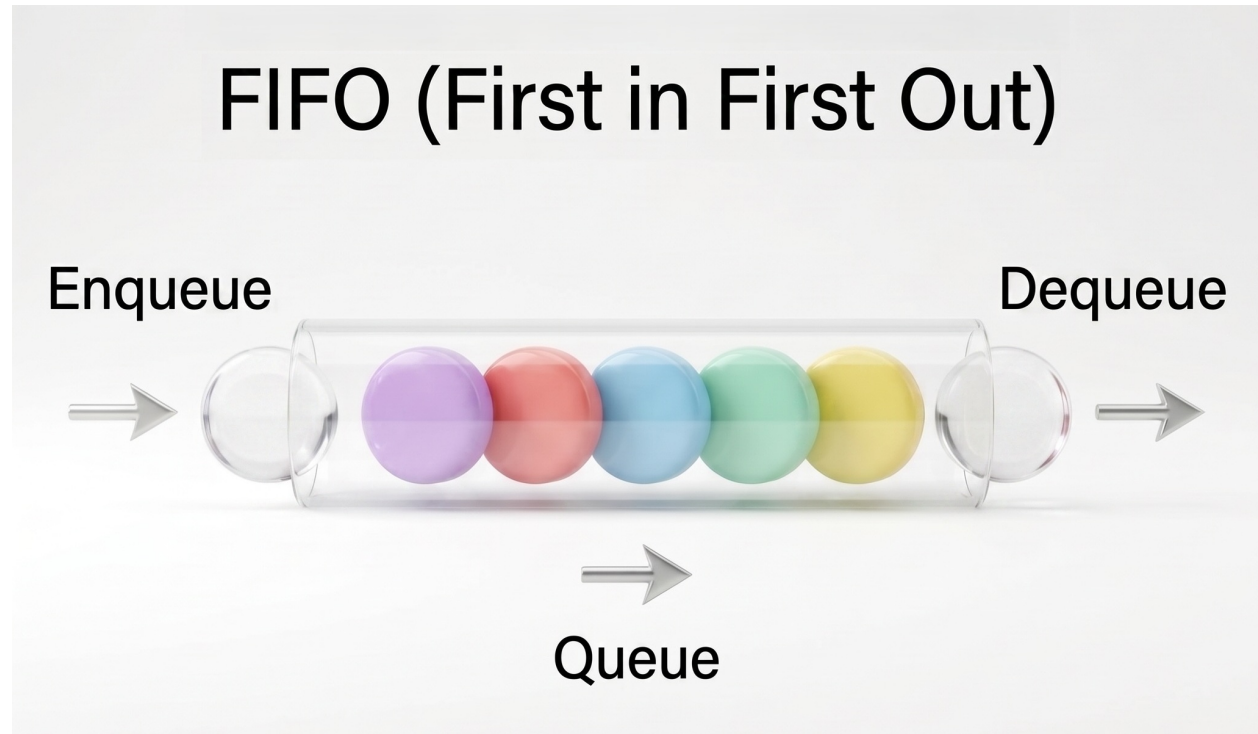
- 1. Project Overview
 - Summary
 - What is a FIFO?
 - Why Asynchronous FIFOs Matter
 - Project Directory Structure
 - 2. FIFO Architecture & Functionality
 - Top-Level Architecture
 - How the FIFO Works
 - 3. Clock Domain Crossing (CDC) & Synchronization Architecture
 - Gray Code Encoding
 - Preventing Metastability (Synchronizers)
 - Synchronization Latency & Safe Flag Pessimism
 - 4. UVM Verification Environment
 - Why a Simple Testbench is Insufficient
 - UVM Testbench Architecture
 - Testbench Signal Map
 - 5. SVA & Coverage
 - Why use SystemVerilog Assertions?
 - SystemVerilog Assertions - 13 Safety Checkers
 - Defense in Depth (Data Integrity)
 - Functional & Structural Code Coverage
 - Verification Trace Matrix (Features Verified)
 - Coverage Results
 - 6. Waveform Analysis & Simulation Guide
 - Read & Write Transactions
 - Quick Start & Simulation Guide
-

1. Project Overview

Summary

This project implements a parameterized dual-clock asynchronous FIFO in VHDL for safe data transfer across independent clock domains (CDC). The design is fully verified using a SystemVerilog UVM environment with SVA assertions and 100% coverage.

What is a FIFO?



A FIFO (First-In, First-Out) is a hardware data buffer where the first piece of data written into the memory is the first piece of data read out, acting like a queue. An **asynchronous** FIFO has independent clocks for writing and reading, allowing systems operating at different speeds to communicate.

Why Asynchronous FIFOs Matter

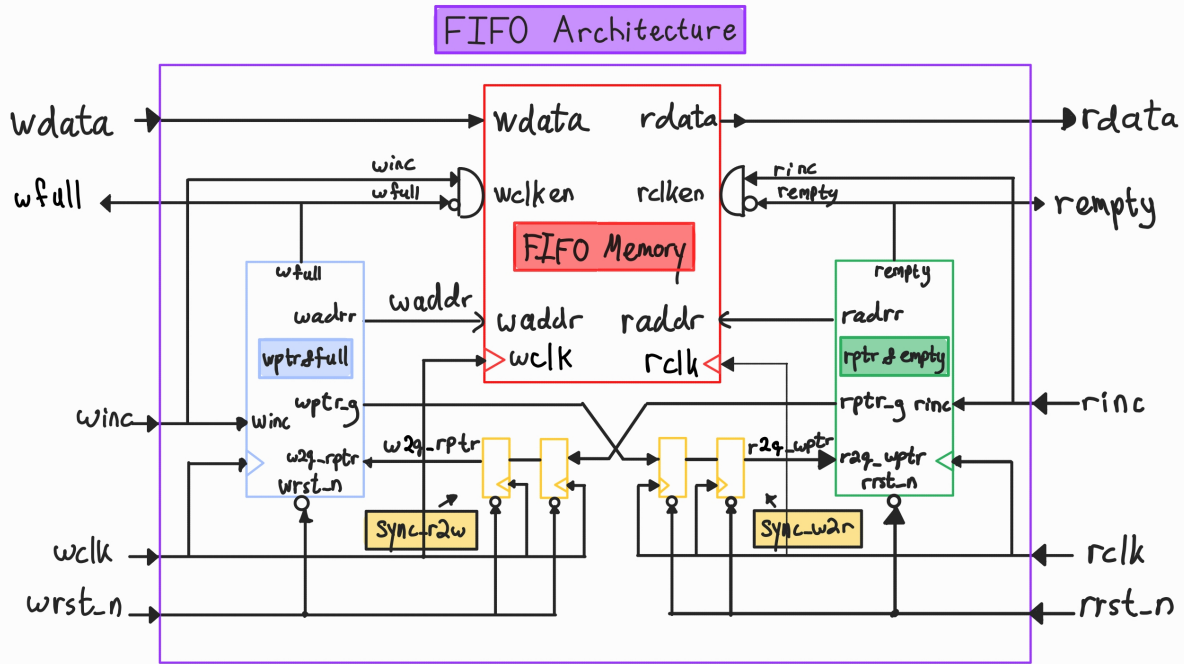
Modern digital chips operate different modules with independent clock frequencies. Asynchronous FIFOs serve as critical data bridges between these clock domains, ensuring fast and reliable data transfer without data loss or corruption.

Project Directory Structure

Directory	Language / Environment	Purpose & Key Contents
rtl/	VHDL-2008	Dual-clock FIFO core (fifo.vhd), memory array, Gray pointers, and 2FF synchronizers
uvm/	SystemVerilog (UVM 1.2)	Complete UVM verification suite (Agents, Drivers, Monitors, Sequences, SVA, Scoreboard)
sim/	TCL / PowerShell / Batch	Automated simulation runner scripts (run.do , run.ps1 , run.bat)

2. FIFO Architecture & Functionality

Top-Level Architecture



How the FIFO Works

An asynchronous FIFO acts as a temporary queue for transferring data between two independent clock domains (Write domain and Read domain):

1. **Writing Data:** When a write is requested and the buffer is not full, data is written into memory at the write address, and the write pointer increments.
2. **Reading Data:** When a read is requested and the buffer is not empty, data is retrieved from memory at the read address, and the read pointer increments.
3. **Full & Empty Status:**
 - **Empty:** Triggered when the read pointer catches up to the synchronized write pointer (all data has been read out).
 - **Full:** Triggered when the write pointer wraps around and catches up to the synchronized read pointer (buffer is completely filled).
4. **CDC Pointer Transfer:** Pointers are converted to Gray code and passed through 2-stage flip-flop (2FF) synchronizers, allowing safe cross-domain comparison without sampling skew or metastability.

3. Clock Domain Crossing (CDC) & Synchronization Architecture

Asynchronous clock domain crossings (CDC) pose two fundamental hardware challenges: **multi-bit pointer skew** and **metastability**. Below is a detailed breakdown of how this FIFO design mitigates both risks: Section 3.1 addresses pointer skew via Gray code encoding, and Section 3.2 resolves metastability via dual-stage flip-flop synchronizers.

3.1 Gray Code Encoding

Problem: Multi-Bit Skew Corruption

In a clock domain crossing, transferring multi-bit pointers is dangerous due to **wire/routing skew** and differences in path delays.

When a binary pointer increments, multiple bits often change simultaneously. For example, when a 3-bit binary pointer transitions from 3 (011) to 4 (100), all three bits must toggle. Due to routing skew, these bit transitions arrive at destination flip-flops at slightly different times. If the destination clock samples the pointer mid-transition, it can capture arbitrary intermediate corrupted values, leading to false full/empty flag assertions or corrupted pointer logic.

Current Binary State	Next Binary State	Sampled Intermediate Values
011 (3)	100 (4)	111 (7), 000 (0), 001 (1), 101 (5), 110 (6), 010 (2)

Solution: Single-Bit Toggle (Gray Code)

To guarantee CDC safety, pointer values are converted to **Gray Code** prior to domain crossing. In Gray code, consecutive numerical values differ by **exactly one bit**.

$$\text{Binary } 3 \rightarrow 4 \iff \text{Gray } 010 \rightarrow 110$$

Since only a single bit toggles during pointer increment:

- There are **no multi-bit intermediate or transient states** to sample.
- If the receiving clock samples the signal precisely during a bit transition, the sampled result will resolve to either the **previous state** (010) or the **new state** (110).
- Both are valid, coherent pointer states. Sampling the old value simply delays flag assertion by one destination clock cycle (safe pessimism) without corrupting memory management logic.

Implementation: Binary-to-Gray Converter

$$G = B \text{ xor } (B \gg 1)$$

3.2 Preventing Metastability (Synchronizers)

Problem: Setup/Hold Timing Violations

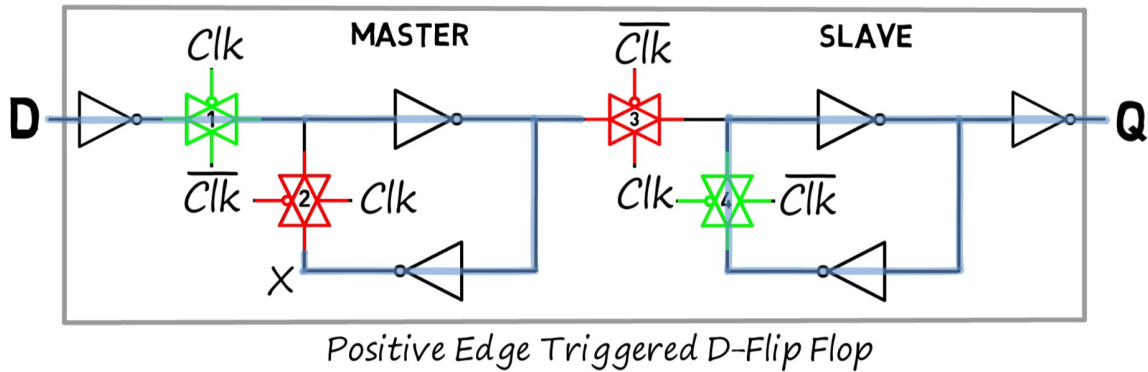
Even with Gray-coded pointers, a single transitioning bit can still violate the setup or hold timing requirements of the receiving flip-flop, leading to **metastability**.

Flip-flops require input signals to remain stable during a setup and hold window around the active clock edge. Because the clocks are asynchronous, such violations cannot be completely avoided.

How Metastability Occurs — Internal Flip-Flop Dynamics A flip-flop can be viewed internally as a regenerative circuit containing cross-coupled inverters controlled by transmission gates.

1. **Before the Clock Edge:** The input data propagates through the input stage and establishes a voltage on an internal storage node.

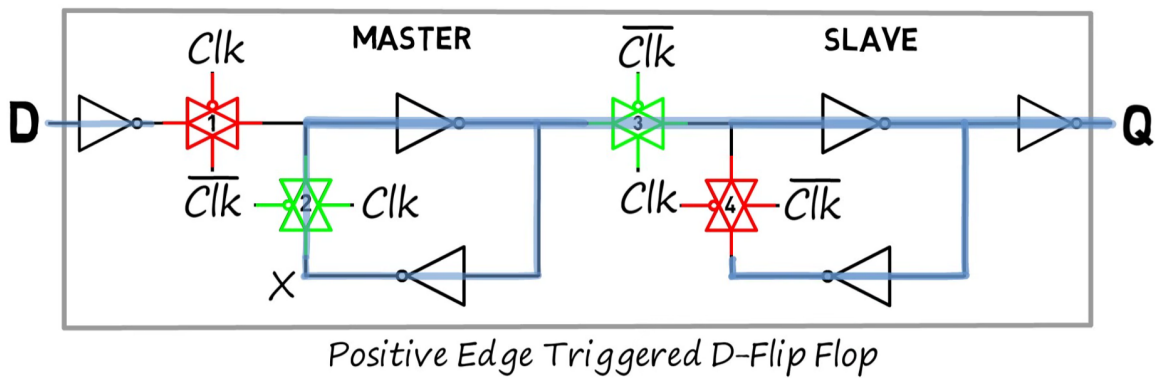
Flip Flop



Clk

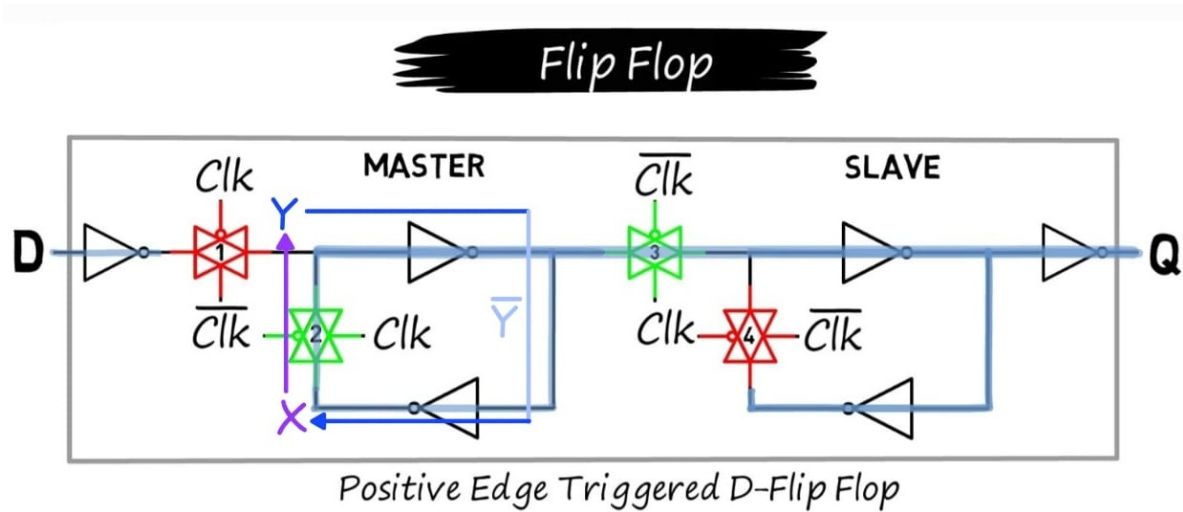
2. **At the Clock Edge:** The transmission gates change state, disconnecting the input path and enabling the regenerative feedback path. If the input changes within the setup/hold window, the internal nodes may contain conflicting voltage levels.

Flip Flop



Clk

3. **Analog Contention:** At this point, the internal nodes are no longer behaving as ideal digital 0/1 signals. The feedback path and the transmission-gate paths interact dynamically. One node can drive the other through the feedback inverters while the other node simultaneously affects the first. The exact transient behavior depends on transistor drive strengths, capacitances, propagation delays, and process variations.



4. **Metastable Equilibrium:** The cross-coupled inverter pair has two stable states (logic 0 and logic 1) and an unstable equilibrium point between them. This equilibrium voltage, often denoted V_M , is typically near $V_{DD}/2$, but is not necessarily exactly $V_{DD}/2$.

If the contention leaves the regenerative loop sufficiently close to this unstable equilibrium, the positive feedback initially has only a very small voltage difference to amplify. The circuit therefore requires additional time to resolve to a valid logic 0 or 1.

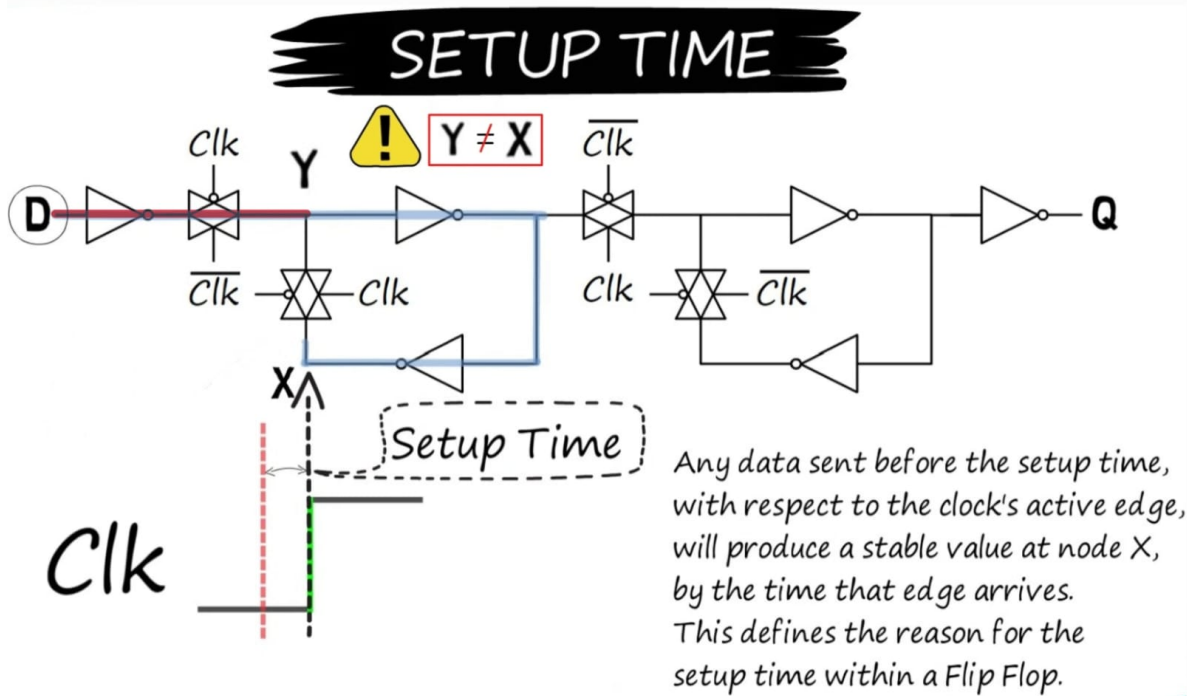
5. **Regeneration:** Around the metastable point, the voltage difference can grow approximately exponentially:

$$\Delta V(t) = \Delta V_0 e^{t/\tau}$$

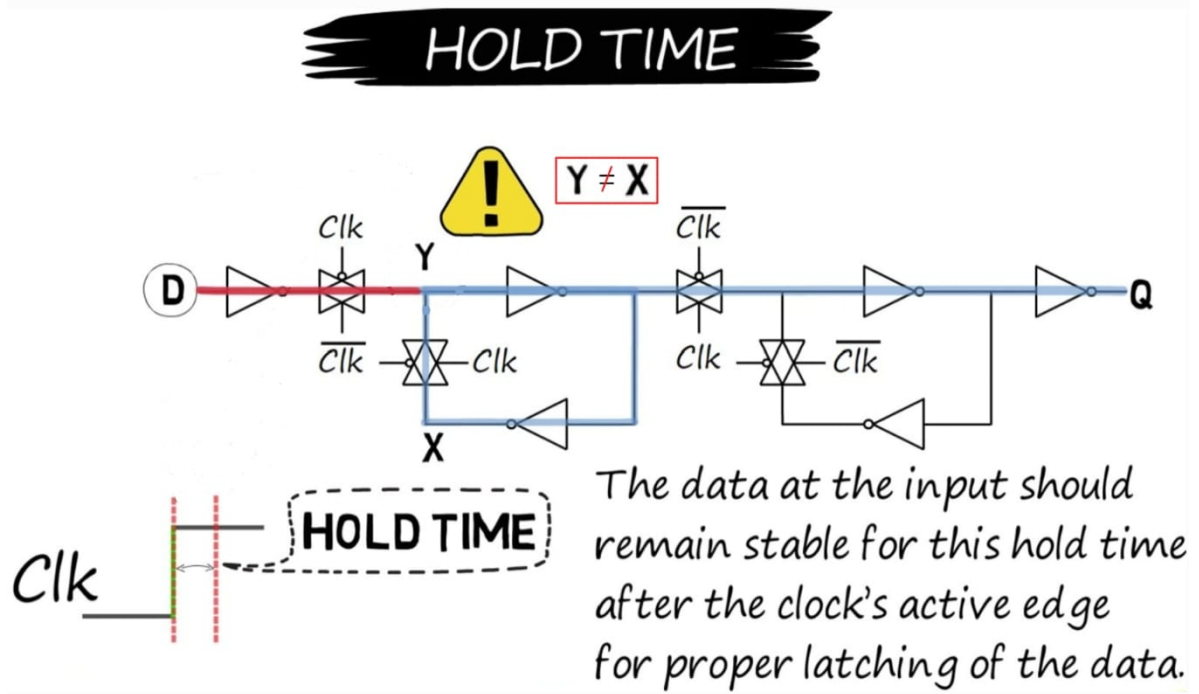
where ΔV_0 is the initial voltage difference from the metastable equilibrium and τ is the characteristic resolution time constant of the circuit.

Therefore, the closer the internal state is to the unstable equilibrium after the initial transient, the longer the resolution can take. The important quantity is not whether a node is exactly at $V_{DD}/2$, but how close the regenerative circuit as a whole is to its unstable equilibrium.

6. **Setup Violation:** Occurs if input D transitions too close to the clock edge. Although D propagates through the input transmission gate to node Y , it fails to propagate through the internal inverters in time to establish a stable, matching voltage on node X before sampling. When the clock edge arrives and the feedback transmission gate closes while $Y \neq X$, the conflicting voltages fight each other, trapping the regenerative latch near its unstable equilibrium and causing metastability.



7. **Hold Violation:** Occurs if input D changes too soon after the clock edge. Because the input transmission gate requires finite time to fully shut off, a premature transition on D leaks into node Y while the feedback transmission gate is already closing. This disrupts the closing feedback loop ($Y \neq X$), disturbing the settling state and potentially forcing the regenerative latch back toward metastability.

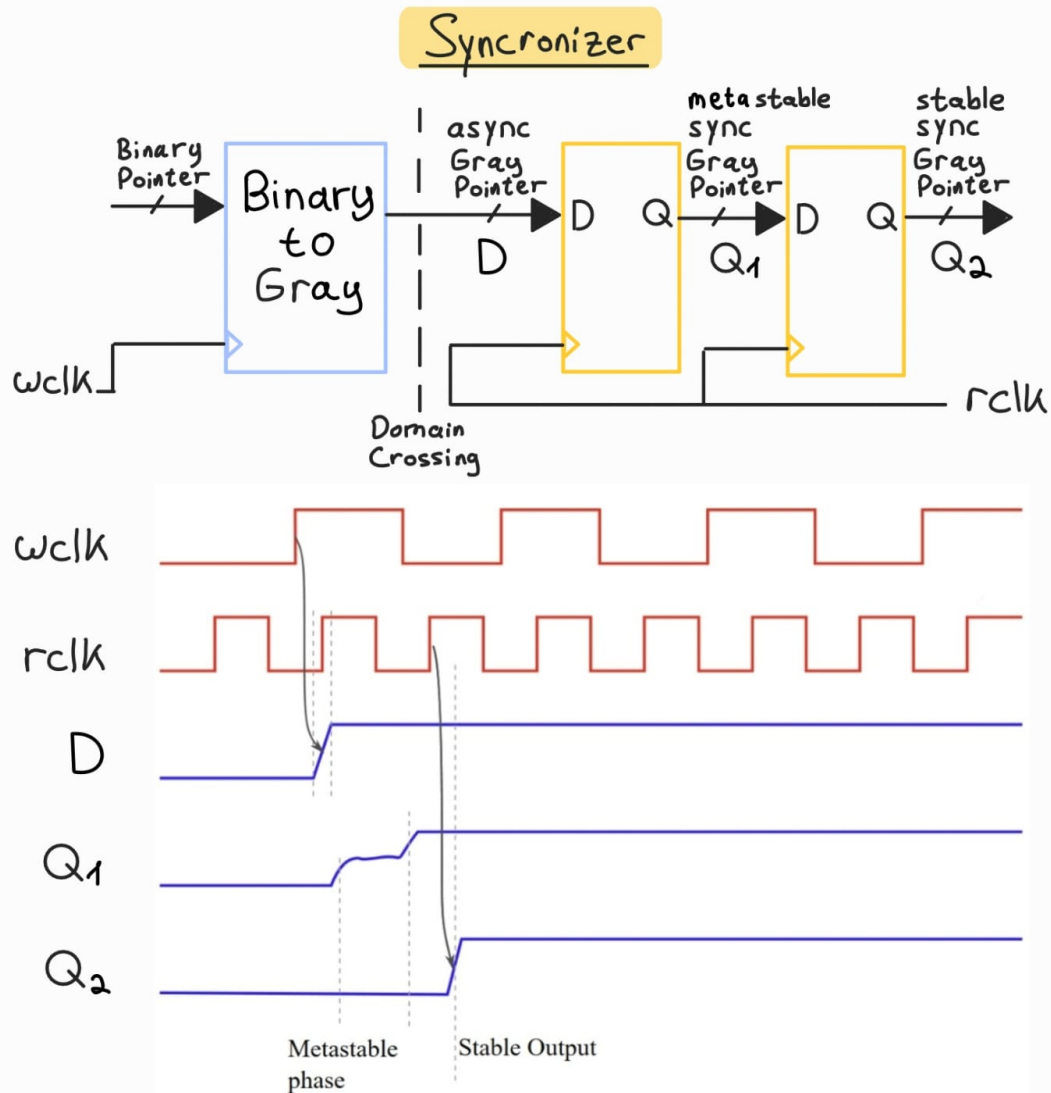


Key Point Metastability occurs when a timing violation leaves internal nodes in conflict ($Y \neq X$), closing the regenerative feedback loop forces the latch close to its unstable equilibrium point (V_M). The closer the

initial voltage difference (ΔV_0) is to zero, the longer positive feedback requires to amplify the signal and resolve the flip-flop to a valid digital '0' or '1'.

Solution: Synchronizers (2FF)

To prevent metastable outputs from propagating into internal control logic, Gray-coded pointers pass through **2-Stage Flip-Flop Chains** granting a full clock cycle of resolution time for any metastable state to decay



1. **Stage 1 (Q_1):** Samples the raw asynchronous Gray pointer. If a setup/hold violation occurs, Q_1 may become metastable.
2. **Resolution Window (t_r):** Q_1 is given approximately one destination-clock cycle for any metastable condition to resolve through the regenerative feedback into a deterministic digital '0' or '1'.
3. **Stage 2 (Q_2):** Samples the settled output of Q_1 . Because the probability of Q_1 remaining metastable for a full clock cycle is exponentially low, Q_2 creates a clean, synchronized signal.

Mean Time Between Failures (MTBF) The reliability of a synchronizer chain is quantified by its MTBF, which increases exponentially with the resolution time (t_r):

$$\text{MTBF} = \frac{e^{s \cdot t_r}}{T_0 \cdot f_{\text{clk}} \cdot f_{\text{data}}}$$

Parameter	Description
t_r	Available settling time ($T_{\text{clk}} - t_{su}$)
s	Resolution time constant of the process node
T_0	Deep-metastability aperture factor
$f_{\text{clk}} / f_{\text{data}}$	Destination clock frequency and incoming Gray pointer toggle rate

The insertion of Q_2 expands t_r by nearly an entire clock period, which can boost MTBF from seconds to years.

3.3 Synchronization Latency & Safe Flag Pessimism

Passing pointers across asynchronous clock domains via 2FF synchronizers introduces a **2 clock cycle delay**. Because pointers are compared against slightly delayed values from the opposite domain, status flags operate with a **safe, pessimistic bias**:

- **wfull Pessimism:** Compares write pointer to a read pointer that is 2 cycles old. The write domain may see the FIFO as "Full" slightly longer than it actually is (if reads occurred during synchronization).
 - **Safety Impact:** Prevents **overflow** (overwriting data), at the cost of a temporary pause in write throughput.
- **rempty Pessimism:** Compares read pointer to a write pointer that is 2 cycles old. The read domain may see the FIFO as "Empty" slightly longer than it actually is (if writes occurred during synchronization).
 - **Safety Impact:** Prevents **underflow** (reading garbage data), at the cost of a 2-cycle latency delay before newly written data can be read out.

4. UVM Verification Environment

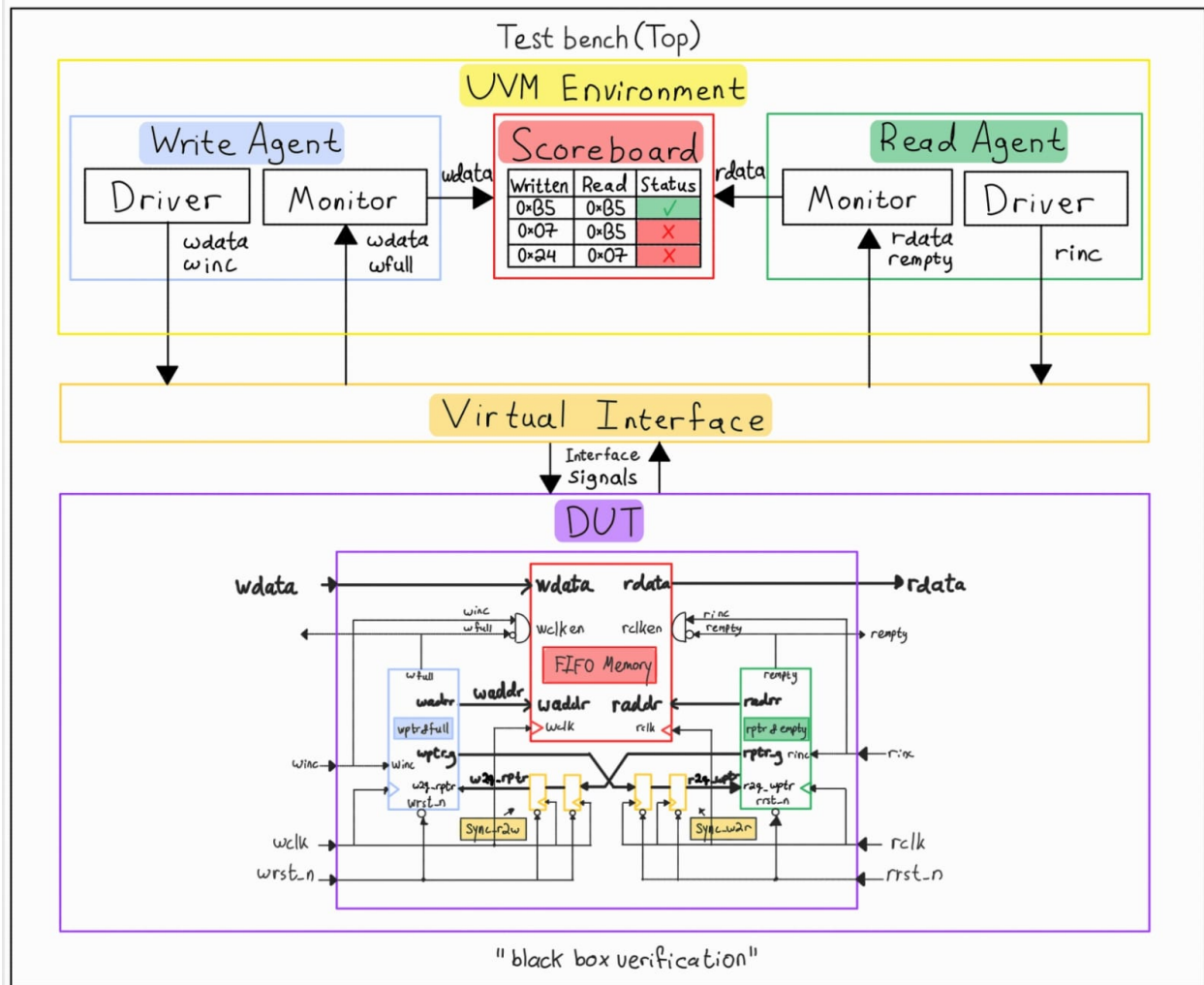
Why a Simple Testbench is Insufficient

A standard testbench focuses on toggling individual pins, making it difficult to handle the multiple clock domains and unpredictable timing of an asynchronous FIFO. To avoid debugging race conditions in the test code itself, we use SystemVerilog UVM to elevate the abstraction from "wiggling pins" to "sending transactions".

The Core Advantages of UVM:

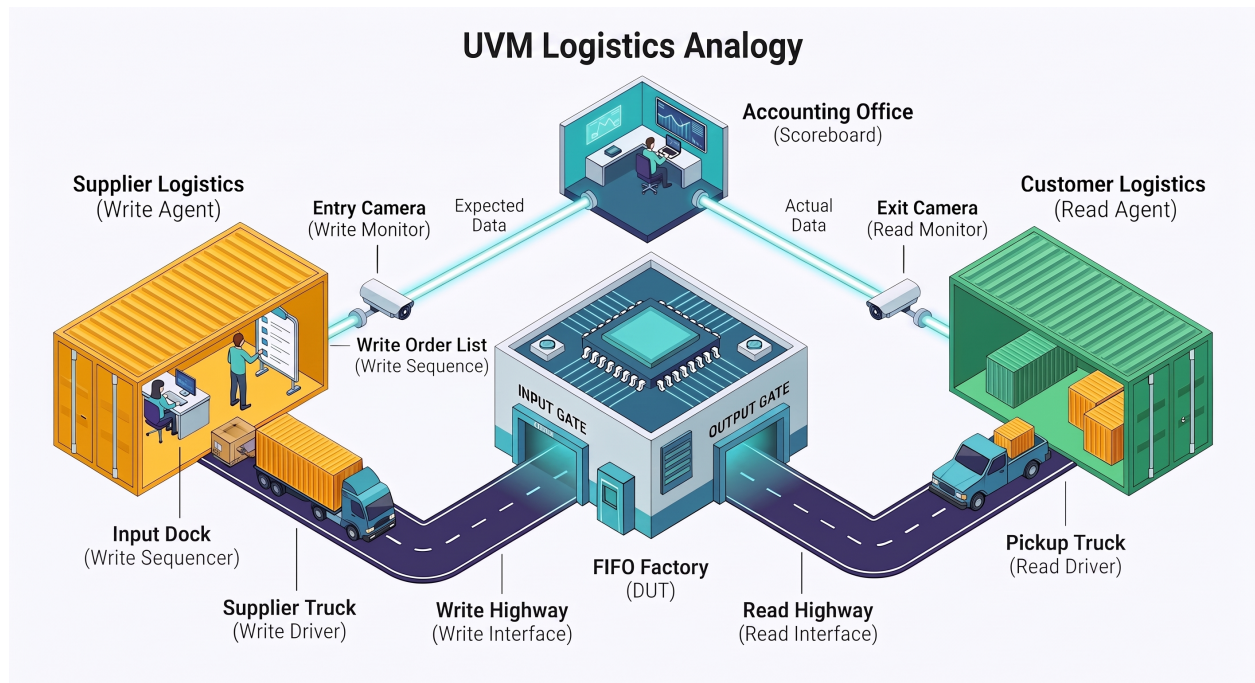
1. **Modularity:** UVM strictly isolates data generation, pin driving, and result checking into different components. This allows us to easily change one component without affecting the others.
2. **Standardized Communication:** UVM libraries provide Transaction Level Modeling (TLM). Standardizing communication guarantees data is passed safely between testbench components without race conditions.
3. **Scalability:** Scaling vertically or horizontally is trivial. We can easily add more FIFOs into the same testing environment by copying the agents and connecting them to the new fifo. Alternatively, we can integrate this FIFO into a larger SoC, by plugging the testbench into a larger verification environment without modification.

UVM Testbench Architecture



The testbench operates using five primary UVM components:

- **Agents:** The managers of a specific clock domain. They encapsulate the data generation, driving, and monitoring activity for their respective sides of the FIFO.
- **Drivers:** Drive DUT input signals.
- **Monitors:** Monitor hardware signals and send them to the scoreboard.
- **Scoreboard:** Records data pushed into the FIFO and compares it against data popped out to verify zero data corruption.
- **Virtual Interface:** Allows the testbench to drive and monitor signals on the DUT's pins.



Beyond these five primary components, the testbench relies on several smaller elements like sequences and transactions to function. Below is an explanation of how these parts work together to verify the FIFO, using a shipping container logistics analogy:

Analogy	UVM Concept	Role in FIFO Verification
[Data Packet] Write Container	Write Transaction	Inbound data payload (<code>wdata</code>) driven into the FIFO.
[Data Packet] Read Container	Read Transaction	Outbound data payload (<code>rdata</code>) popped from the FIFO.
[Order List] Write Order List	Write Sequence	Generates planned write traffic (e.g., burst writes until <code>wfull</code>).
[Order List] Read Order List	Read Sequence	Generates planned read requests (e.g., burst reads until <code>rempty</code>).
[Driver Depot] Input Dock	Write Sequencer	Queues write containers for the supplier truck.
[Driver Depot] Output Dock	Read Sequencer	Queues read request triggers for the pickup truck.
[Transport] Supplier Truck	Write Driver	Drives write signals (<code>winc</code> , <code>wdata</code>) onto the write interface.
[Transport] Pickup Truck	Read Driver	Drives read request signals (<code>rinc</code>) onto the read interface.
[Highway] Supplier Logistics	Write Agent	Groups Write Sequencer, Driver, and Monitor for the write clock domain (<code>wclk</code>).
[Local Road] Customer Logistics	Read Agent	Groups Read Sequencer, Driver, and Monitor for the read clock domain (<code>rclk</code>).
[Toll Checkpoint] Write Highway	Write Interface	Write clock domain wires (<code>wclk</code> , <code>wrst_n</code> , <code>winc</code> , <code>wdata</code> , <code>wfull</code>).
[Toll Checkpoint] Read Highway	Read Interface	Read clock domain wires (<code>rclk</code> , <code>rrst_n</code> , <code>rinc</code> , <code>rdata</code> , <code>rempty</code>).
[Audit Camera] Entry Camera	Write Monitor	Scans inbound containers entering the FIFO and sends expected logs to Accounting Office.

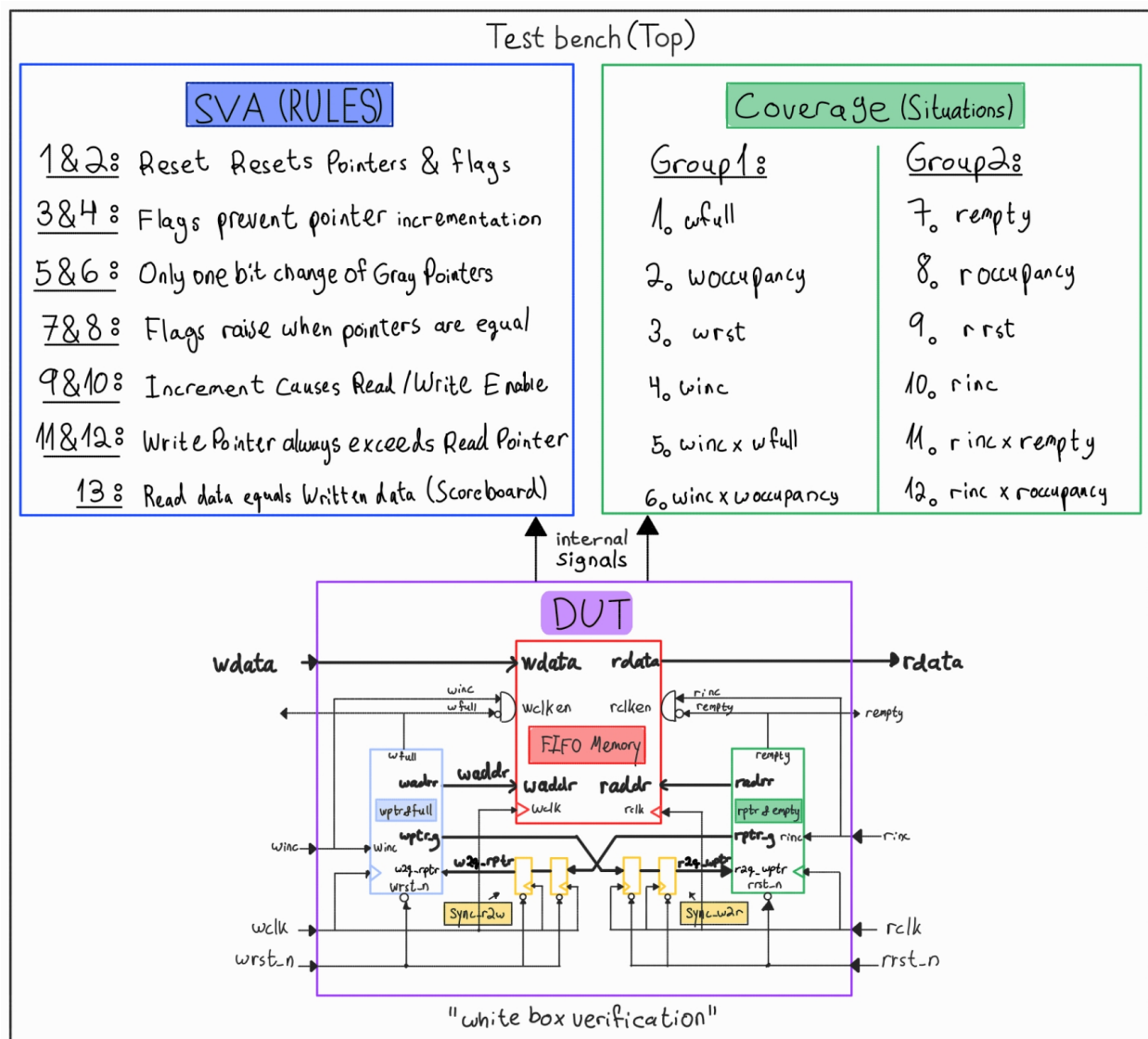
Analogy	UVM Concept	Role in FIFO Verification
[Audit Camera] Exit Camera	Read Monitor	Scans outbound containers leaving the FIFO and sends actual logs to Accounting Office.
[Quality Control] Accounting Office	Scoreboard	Compares Expected (Write) vs Actual (Read) containers to verify data integrity (PASS/FAIL).

Testbench Signal Map

To get into the specifics, below is a table that explains how each hardware signal is influenced by each component in the UVM testbench.

Signal	Declared In	Driven By	Sampled By
wclk, rclk	uvm/fifo_top.sv	fifo_top (always statement)	DUT, Interface Clocking Blocks
wrst_n, rrst_n	uvm/fifo_if.sv	fifo_top (Initial Time-0) & fifo_test / fifo_reset_recovery_test (Mid-traffic reset)	DUT, Drivers, Monitors, SVA
winc, wdata	uvm/fifo_if.sv	fifo_w_driver	DUT, fifo_w_monitor , SVA
rinc	uvm/fifo_if.sv	fifo_r_driver	DUT, fifo_r_monitor , SVA
wfull, rempty, rdata	uvm/fifo_if.sv	FIFO DUT	Drivers, Monitors, Scoreboard, Test cases
wclken_wire, rclken_wire	uvm/fifo_if.sv	FIFO DUT (via fifo_connector bridge)	Monitors, SVA

5. SVA & Coverage



Why use SystemVerilog Assertions?

We use the **Scoreboard** as our **Black-Box** checker. Operating in the software domain, it uses **Object-Oriented Programming** to abstract external pin activity into high-level, "timeless" transactions. Because it tracks abstract data rather than physical clocks, the Scoreboard is blind to timing bugs but perfect for verifying data that stays in the queue over long periods of time.

However, if the Scoreboard detects a failure, it only tells us that the data broke, not *why*. Did a write pointer increment too fast? Did a synchronizer fail?

To catch these hardware-level bugs the exact cycle they happen, we implemented **SVA** as our **White-Box** checker. Built using strictly timed **Temporal Logic**, SVA lives in the hardware domain. Unlike the Scoreboard, SVA is perfect for checking immediate, short-term events. It monitors internal signals cycle-by-cycle, telling us exactly *why* and *when* a protocol was violated.

SystemVerilog Assertions - 13 Safety Checkers

The verification environment contains **13 concurrent and immediate SystemVerilog Assertions** to check the safety and correctness of the FIFO protocol:

1. **Write Domain Reset:** Asserts that when the write reset is active, all write pointers (`waddr`, binary, Gray) and the full flag are correctly initialized to 0.
2. **Read Domain Reset:** Asserts that when the read reset is active, all read pointers (`raddr`, binary, Gray) are initialized to 0 and the empty flag is asserted (1).
3. **Write Pointer Stability (No Overwrite):** Verifies that if the full flag is asserted, any subsequent write requests (`winc`) will not increment the write pointers.
4. **Read Pointer Stability (No Overread):** Verifies that if the empty flag is asserted, any subsequent read requests (`rinc`) will not increment the read pointers.
5. **Write Gray-Code Integrity:** Checks that the write Gray-code pointer changes by only one bit per increment step.
6. **Read Gray-Code Integrity:** Checks that the read Gray-code pointer changes by only one bit per increment step.
7. **Empty Flag Generation:** Verifies that when the read Gray pointer equals the synchronized write Gray pointer, the empty flag is asserted.
8. **Full Flag Generation:** Verifies that when the write Gray pointer matches the synchronized read Gray pointer (with the two MSBs inverted), the full flag is asserted.
9. **Write Enable Gating:** Verifies that `wclken` is only active when `winc = 1` and `wfull = 0`.
10. **Read Enable Gating:** Verifies that `rclden` is only active when `rinc = 1` and `rempty = 0`.
11. **Write Memory Occupancy Bound Check:** Verifies that the difference between the write pointer and synchronized read pointer never exceeds the FIFO depth.
12. **Read Memory Occupancy Bound Check:** Verifies that the difference between the synchronized write pointer and read pointer never exceeds the FIFO depth on the read side.
13. **Data Integrity Checker:** Compares the read data against a golden shadow memory array (`fifo_data`) at the read address to check for data corruption.

Defense in Depth (Data Integrity)

While SVA checker #13 behaves like a scoreboard by verifying data integrity, keeping both provides **Defense in Depth** for debugging.

Why check data integrity twice? If both the SVA and the Scoreboard fail, the internal memory array corrupted the data. If *only* the Scoreboard fails, the memory is fine, but data was corrupted externally on the bus.

Functional & Structural Code Coverage

The verification environment combines **Structural Code Coverage** on the RTL design units with **Functional and Assertion Coverage** defined via SystemVerilog covergroups and SVA.

Coverage Goals & Coverpoints

The verification plan defines **12 distinct coverpoints and crosses** (split between the write-side and read-side covergroups) to verify correct operation across all FIFO scenarios:

Write-Side Covergroup (`fifo_w_cg`)

1. **wfull:** Coverage of the FIFO Full flag states (Asserted vs. Deasserted).
2. **w_occupancy:** Tracks write-side occupancy levels (`empty`, `almost_empty`, `mid_range`, `almost_full`, `full`).
3. **winc:** Verifies write command request activation.

4. **wrst_n**: Verifies write domain reset states.
5. **Cross winc × wfull**: Proves write attempts are simulated under both Full and Non-Full conditions (tests overflow protection).
6. **Cross winc × w_occupancy**: Proves write requests are executed at every possible FIFO fill level.

Read-Side Covergroup (fifo_r_cg)

7. **rempty**: Coverage of the FIFO Empty flag states (Asserted vs. Deasserted).
8. **r_occupancy**: Tracks read-side occupancy levels (empty, almost_empty, mid_range, almost_full, full).
9. **rinc**: Verifies read command request activation.
10. **rrst_n**: Verifies read domain reset states.
11. **Cross rinc × rempty**: Proves read attempts are simulated under both Empty and Non-Empty conditions (tests underflow protection).
12. **Cross rinc × r_occupancy**: Proves read requests are executed at every possible FIFO fill level.

Verification Trace Matrix (Features Verified)

Verification Scenario	Status	SVA Checker Evidence	Functional Coverage Evidence
Reset Recovery	✓ Passed	SVA #1, #2 (Write/Read Domain Reset)	Coverpoints wrst_n / rrst_n
Pointer Stability (Overflow/Underflow)	✓ Passed	SVA #3, #4 (Write/Read Pointer Stability)	Crosses winc × wfull / rinc × rempty
Gray Pointer Correctness	✓ Passed	SVA #5, #6 (Gray-Code Integrity)	Indirectly by Coverpoints w_occupancy / r_occupancy
FIFO Flag Detection (Empty/Full)	✓ Passed	SVA #7, #8 (Flag Generation & CDC Sync)	Coverpoints rempty / wfull
Enable Logic Gating	✓ Passed	SVA #9, #10 (Write/Read Enable Gating)	Crosses winc × wfull / rinc × rempty
Pointer Wraparound & Bounds	✓ Passed	SVA #11, #12 (Memory Occupancy Bound Checks)	Coverpoints w_occupancy / r_occupancy
Data Integrity	✓ Passed	SVA #13 (Data Integrity Checker)	Verified by UVM Scoreboard

Coverage Results

Coverage Type	Target	Status	Result
Functional Coverage	Covergroups / Coverpoints	✓ Covered	100%
Assertion Coverage	SystemVerilog Assertions	✓ Covered	100%
RTL Code Coverage	VHDL Design Units	✓ Covered	100%

- **Functional Coverage**: Proves the testbench successfully hit all defined design features and edge cases (e.g., full, empty, write while full).
- **Assertion Coverage**: Confirms all safety checkers (SVA) protecting against overflow, underflow, and logic faults were active and passed.
- **RTL Code Coverage (Structural)**: Verifies the simulator executed every physical line, branch path, boolean expression, and signal toggle in the hardware code.

The screenshot displays the ModelSim Coverage Report Summary interface. It features a dark blue header bar with the title "ModelSim Coverage Report Summary". Below the header, there are two main panels, each with a green title bar indicating 100% coverage.

Instance Coverage Summary (100%)

Coverage Type ↑	Bins	Hits	Misses	Coverage
Search...	Search...	Search...	Search...	Search...
Assertions	12	12	0	100%
Branches	31	31	0	100%
Conditions	7	7	0	100%
Covergroups	2	na	na	100%
Coverpoints/Crosses	12	na	na	na
Coverage Bins	50	50	0	100%
Expressions	8	8	0	100%
Statements	94	94	0	100%
Toggles	460	460	0	100%

Design Units Coverage Summary (100%)

Coverage Type ↑	Bins	Hits	Misses	Coverage
Search...	Search...	Search...	Search...	Search...
Assertions	12	12	0	100%
Branches	29	29	0	100%
Conditions	7	7	0	100%
Covergroups	2	na	na	100%
Coverpoints/Crosses	12	na	na	na
Coverage Bins	50	50	0	100%
Expressions	8	8	0	100%
Statements	90	90	0	100%
Toggles	602	602	0	100%

Total Coverage (100%)

Overall Design Unit Coverage Summary:

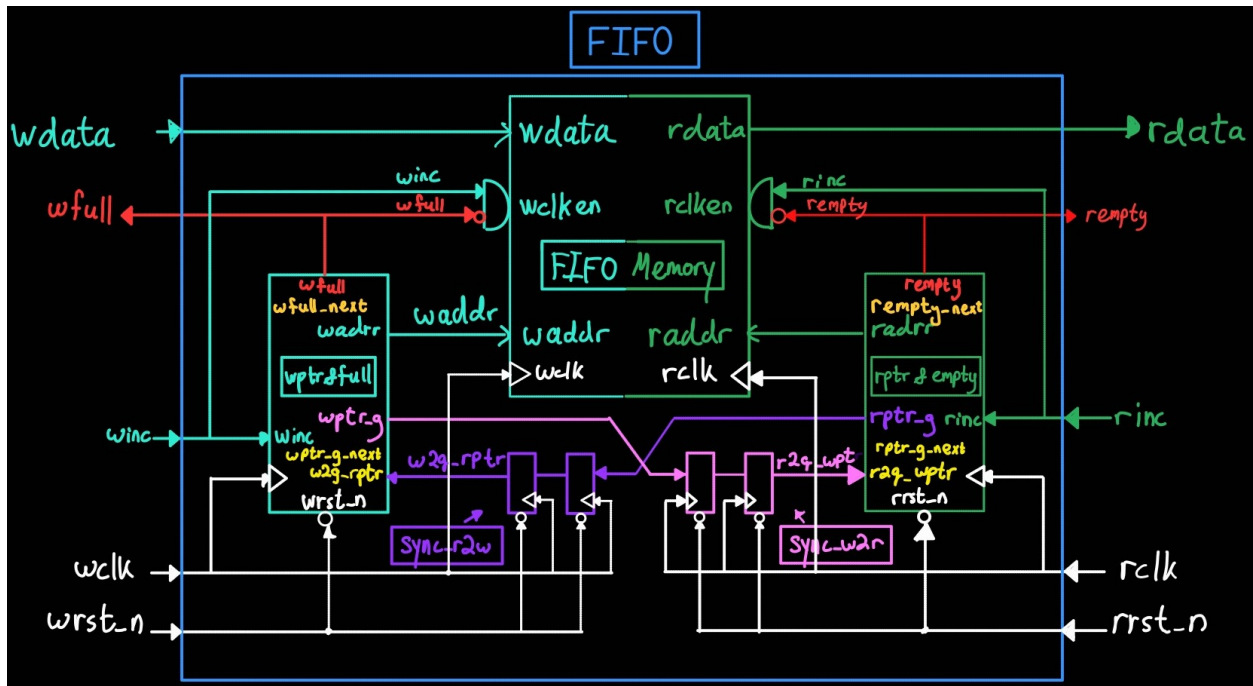
Assertions	Branches	Conditions	Expressions	Covergroups	Statements	Toggles
100%	100%	100%	100%	100%	100%	100%

Coverage Summary by Design Unit:

Design Unit ↑	Assertions	Branches	Conditions	Expressions	Covergroups	Statements	Toggles	Total
Search... ▼	Search... ▼	Search... ▼	Search... ▼	Search... ▼	Search... ▼	Search... ▼	Search... ▼	Search... ▼
work.fifo(fifo_arc)	-	-	-	100%	-	100%	100%	100%
work.fifo_connector	-	-	-	-	-	-	100%	100%
work.fifo_if	-	-	-	-	-	100%	100%	100%
work.fifo_mem(fifo_mem_arc)	-	100%	-	-	-	100%	100%	100%
work.fifo_r_ptr(fifo_r_ptr_arc)	-	100%	100%	-	-	100%	100%	100%
work.fifo_sva	100%	100%	-	100%	100%	100%	100%	100%
work.fifo_synchronizer(fifo_synchronizer_arc)	-	100%	-	-	-	100%	100%	100%
work.fifo_top	-	-	-	-	-	100%	100%	100%
work.fifo_w_ptr(fifo_w_ptr_arc)	-	100%	100%	-	-	100%	100%	100%

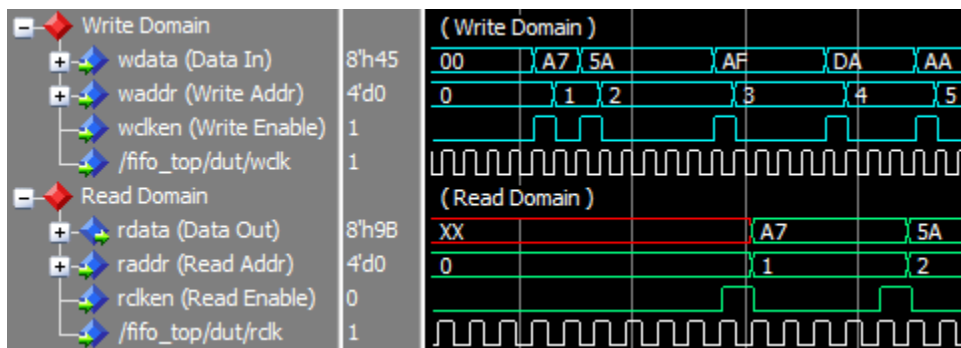
HlsC		HlsC	
Clocks & Resets clk clk clk clk_rstn clk clk_rstn		clk clk_rstn	clk clk_rstn
Write Domain write_data_in write_data_in write_data_in write_data_in		write_data_in write_data_in write_data_in write_data_in	write_data_in write_data_in write_data_in write_data_in
Read Domain read_data_out read_data_out read_data_out read_data_out		read_data_out read_data_out read_data_out read_data_out	read_data_out read_data_out read_data_out read_data_out
Write Pointers: wful proof write_ptr_next write_ptr_next write_ptr_next write_ptr_next		write_ptr_next write_ptr_next write_ptr_next write_ptr_next	write_ptr_next write_ptr_next write_ptr_next write_ptr_next
Read Pointers: empty proof read_ptr_next read_ptr_next read_ptr_next read_ptr_next		read_ptr_next read_ptr_next read_ptr_next read_ptr_next	read_ptr_next read_ptr_next read_ptr_next read_ptr_next
Write Pointers: wful proof write_ptr_next write_ptr_next write_ptr_next write_ptr_next		write_ptr_next write_ptr_next write_ptr_next write_ptr_next	write_ptr_next write_ptr_next write_ptr_next write_ptr_next
Read Pointers: empty proof read_ptr_next read_ptr_next read_ptr_next read_ptr_next		read_ptr_next read_ptr_next read_ptr_next read_ptr_next	read_ptr_next read_ptr_next read_ptr_next read_ptr_next

Waveforms Colors Match Diagram Signals



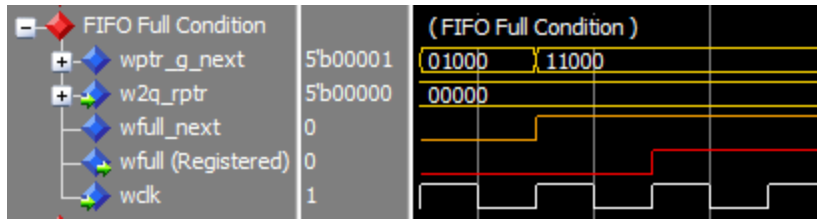
Read & Write Transactions

This waveform demonstrates standard FIFO data path operations. Data is written into the memory array (**wdata**) on the write clock domain when **winc** is asserted. It is subsequently read out (**rdata**) on the read clock domain in a First-In-First-Out sequence. The integrity of the data stream (e.g., A7, 5A, AF) is preserved perfectly as it crosses the asynchronous boundary.



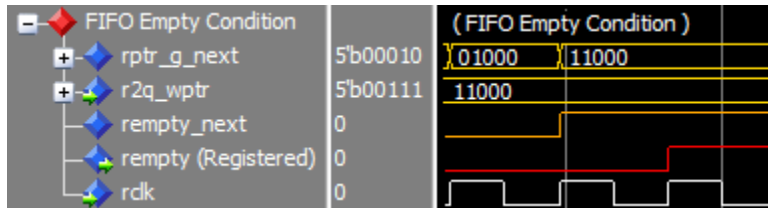
2. FIFO Full Condition

The FIFO asserts **wfull** when the write pointer catches up to the synchronized read pointer. In Gray code, full occurs when pointers are equal with the two Most Significant Bits (MSBs) inverted.



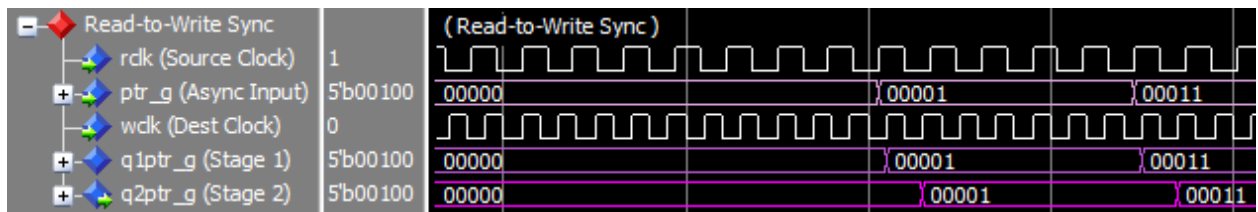
3. FIFO Empty Condition

The FIFO asserts `empty` when the read pointer catches up to the synchronized write pointer (all bits match).

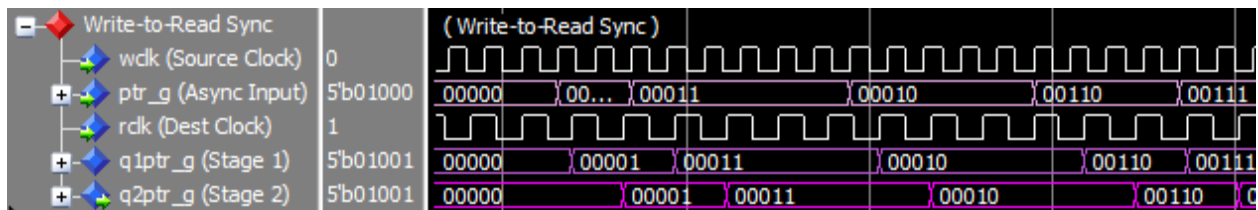


4. Clock Domain Crossing (CDC) Pointer Traces

Read-to-Write Domain Synchronization The read Gray pointer crosses into the write domain through the 2FF synchronizer stages (`q1ptr_g` and `q2ptr_g`) before being evaluated for full condition check.



Write-to-Read Domain Synchronization The write Gray pointer crosses into the read domain through two flip-flop stages before empty condition evaluation.



Quick Start & Simulation Guide

Dynamic Simulation Parameters

You can dynamically configure all 4 simulation parameters at runtime without re-compiling the design:

Default Simulation Parameters

- **Clocks:** Wclk = 5 ns (100 MHz), Rclk = 7 ns (~71.4 MHz)
- **FIFO Bus:** DataWidth = 8 bits, AddrWidth = 4 bits (Depth = 16)

Option 1: Automated Test Runner (Command Line - Quiet Mode)

1. Open terminal and navigate to your `sim` directory:
`cd sim`
 2. Run simulation with default or custom parameters:
1. Run basic test suite (default settings)
`.\run.ps1`

2. Custom Parameters Run
`.\run.ps1 -TestName fifo_reset_recovery_test -Wclk 2 -Rclk 10 -DataWidth 16 -AddrWidth 5`
-

Option 2: Interactive ModelSim GUI (Waveforms)

1. Open ModelSim SE and navigate to your `sim` directory:
`cd sim`
 2. Run simulation with default or custom parameters:
1. Run basic test (default settings)
`do run.do`

2. Custom Parameters Run
`set TESTNAME fifo_reset_recovery_test; set WCLK_HALF 2; set RCLK_HALF 10; set DATA_WIDTH 5;`
`↪ set ADDR_WIDTH 5; do run.do`
-

Author

Shlomo Abrams

Electrical Engineering Student

GitHub